

CHAOS ARENA 2

Documentacion de Clases y API

Total de clases documentadas: 13

Generado automaticamente desde el codigo fuente

*ActividadJuego | BucleJuego | Buff | BuffType | EnemyAI | Event | Juego | Player | ResourceManager | RunManager |
SpriteAnimation | StageDef | VirtualJoystick*

INDICE DE CLASES

1. **ActividadJuego**
Activity principal de Android. Punto de entrada de la aplicacion. Gestiona el ciclo de vida (onCreat...
2. **BucleJuego**
Hilo del bucle de juego. Ejecuta a 60 FPS el ciclo actualizar->renderizar, calcula delta time fijo y...
3. **Buff**
Representa una mejora (buff) que el jugador puede adquirir en modo Arcade. Almacena el tipo, valor n...
4. **BuffType**
Enumeracion de los tipos de mejora disponibles para jugador y enemigos.
5. **EnemyAI**
Inteligencia artificial del personaje CPU. Implementa una maquina de estados (IDLE, CHASE, ATTACK, R...
6. **Event**
Representa un evento aleatorio que ocurre cada 3 combates ganados en Arcade. Puede curar al jugador,...
7. **Juego**
Vista principal del juego (hereda SurfaceView). Contiene toda la logica de pantallas (menu, seleccio...
8. **Player**
Representa a un personaje del juego. Gestiona animaciones por estado (idle, caminar, atacar, danyo, ...
9. **ResourceManager**
Carga y centraliza todos los assets visuales al inicio. Proporciona fuentes (Paint), bitmaps de UI, ...
10. **RunManager**
Gestiona la progresion de una partida en modo Arcade (roguelite): lleva la cuenta de combates ganado...
11. **SpriteAnimation**
Sustituto de TextureAtlas+Animation de libGDX. Carga frames individuales (.png/.webp) desde assets o...
12. **StageDef**
Define un escenario de combate: nombre, bitmap de fondo, offset visual del suelo y la posicion Y de ...
13. **VirtualJoystick**
Joystick tactil dibujado sobre el Canvas. Soporta multitoque (rastrea el puntero especifico que lo a...

ActividadJuego

ActividadJuego.java

Activity principal de Android. Punto de entrada de la aplicación. Gestiona el ciclo de vida (onCreate, onPause, onResume, onDestroy), configura pantalla completa y crea ResourceManager y Juego.

Constantes

Nombre	Valor/Tipo	Descripción
MAX_LEVELS	int = 3	Numero de niveles en modo Historia.
MAX_SLOTS	int = 3	Ranuras de guardado disponibles.
LEVEL_ENEMY_HEALTH	float[]	Vida del enemigo por nivel: {250, 250, 250}.
LEVEL_ENEMY_SPEED	float[]	Velocidad del enemigo por nivel: {300, 400, 500}.
LEVEL_ENEMY_DAMAGE	float[]	Danyo base del enemigo por nivel: {5, 5, 5}.
LEVEL_ATTACK_RATE	float[]	Cadencia de ataque de la IA por nivel (seg): {1.6, 1.0, 0.6}.
LEVEL_ENEMY_NAMES	String[]	Nombres de los enemigos del modo Historia.
CHAR_NAMES	String[]	Personajes seleccionables: Liu Kang, Goro, Ronin.
LEVEL_LORE	String[]	Textos narrativos mostrados antes de cada combate.

Variables y Campos

Nombre	Tipo	Descripción
juego	Juego	Instancia de la vista principal del juego.
resources	ResourceManager	Gestor de assets compartido.
prefs	SharedPreferences	Preferencias persistentes (puntuaciones, mute).

Metodos

Firma / Nombre	Descripción
onCreate(Bundle)	Configura pantalla completa, instancia ResourceManager y Juego.
onPause()	Pausa la musica cuando la Activity pierde el foco.
onResume()	Reanuda la musica al volver a primer plano.
onDestroy()	Libera la musica y recicla los bitmaps de ResourceManager.

BucleJuego

BucleJuego.java

Hilo del bucle de juego. Ejecuta a 60 FPS el ciclo actualizar->renderizar, calcula delta time fijo y salta frames de render si el dispositivo va lento (hasta MAX_FRAMES_SALTADOS) para mantener la logica al dia.

Constantes

Nombre	Valor/Tipo	Descripcion
MAX_FPS	int = 60	Fotogramas por segundo objetivo.
MAX_FRAMES_SALTADOS	int = 5	Maximo de frames que se pueden saltar al ir lentos.
TIEMPO_FRAME	int = 16ms	Duracion ideal de un frame (1000/MAX_FPS).

Variables y Campos

Nombre	Tipo	Descripcion
juego	Juego	Referencia a la logica/render principal.
surfaceHolder	SurfaceHolder	Acceso al canvas de la Surface.
enEjecucion	volatile bool	Flag para parar el bucle desde fuera.
iteraciones	int	Contador total de iteraciones ejecutadas.
tiempoTotal	long	Tiempo acumulado de todas las iteraciones (ms).
maxX / maxY	int	Dimensiones del canvas obtenidas al inicio.

Metodos

Firma / Nombre	Descripcion
BucleJuego(SurfaceHolder, Juego)	Constructor: guarda referencias y lee el tamano inicial del canvas.
run()	Bucle principal: bloquea canvas, llama a actualizar() y renderizar(), duerme el tiempo restante y salta frames si hay retraso.

Buff

Buff.java

Representa una mejora (buff) que el jugador puede adquirir en modo Arcade. Almacena el tipo, valor numerico, si es permanente y la rareza. Contiene el enum interno Rarity (COMMON, RARE, EPIC).

Variables y Campos

Nombre	Tipo	Descripcion
type	BuffType	Tipo de stat que mejora.
value	float	Magnitud de la mejora (porcentaje o valor absoluto).
isPermanent	boolean	Si el buff persiste entre combates (siempre true).
rarity	Rarity	Rareza: COMMON, RARE o EPIC. Multiplica el valor del buff.

Metodos

Firma / Nombre	Descripcion
Buff(BuffType, float, boolean, Rarity)	Constructor que asigna todos los campos.
getDescription()	Devuelve un String con rareza y nombre del buff para la UI.

BuffType

BuffType.java

Enumeracion de los tipos de mejora disponibles para jugador y enemigos.

Constantes

Nombre	Valor/Tipo	Descripcion
DAMAGE	-	Aumenta el multiplicador de danyo del personaje.
SPEED	-	Aumenta el multiplicador de velocidad de movimiento.
HEALTH	-	Aumenta la vida maxima (y la actual en la misma cantidad).
LIFESTEAL	-	Porcentaje de danyo infligido convertido en vida recuperada.
ARMOR	-	Porcentaje de reduccion de danyo recibido.
CRITICAL	-	Probabilidad de que un golpe haga el doble de danyo.
REGEN	-	Puntos de vida recuperados por segundo de forma pasiva.

EnemyAI

EnemyAI.java

Inteligencia artificial del personaje CPU. Implementa una maquina de estados (IDLE, CHASE, ATTACK, RETREAT, COOLDOWN) que decide cada frame si perseguir al jugador, atacar, retroceder o esperar. Incluye reaccion al especial del jugador y seleccion inteligente del tipo de ataque segun distancia y combo.

Variables y Campos

Nombre	Tipo	Descripcion
npc	Player	Personaje que controla la IA.
target	Player	Personaje objetivo (el jugador).
currentState	State	Estado actual de la maquina de estados.
stateTimer	float	Temporizador para transitar al siguiente estado.
attackCooldownTimer	float	Tiempo de espera obligatorio entre ataques.
speed	float	Velocidad de desplazamiento en unidades/seg (default 350).
damage	float	Danyo base de la IA (default 5).
attackRate	float	Segundos entre ataques (default 1.8).

Metodos

Firma / Nombre	Descripcion
EnemyAI(Player npc, Player target)	Constructor: asigna NPC y objetivo.
setSpeed(float)	Cambia la velocidad de movimiento de la IA.
setDamage(float)	Cambia el danyo y actualiza damageMultiplier del Player.
setAttackRate(float)	Cambia el tiempo minimo entre ataques.
getSpeed()	Devuelve la velocidad actual.
getDamage()	Devuelve el danyo actual.
getAttackRate()	Devuelve la cadencia de ataque actual.
update(float delta)	Logica principal: avanza timers, evalua reacciones y ejecuta la maquina de estados.
decideNextState(float)	Elige el siguiente estado basandose en la distancia al jugador y el cooldown.
executeProAttack(float)	Selecciona y ejecuta el tipo de ataque mas adecuado (SPECIAL/KICK/PUNCH).

Event

Event.java

Representa un evento aleatorio que ocurre cada 3 combates ganados en Arcade. Puede curar al jugador, envenenarle, aumentar la dificultad del proximo enemigo o aplicar un modificador de danyo global. Contiene el enum EventType.

Variables y Campos

Nombre	Tipo	Descripcion
type	EventType	Tipo: HEAL, HARDER_COMBAT, GLOBAL_MODIFIER, POISON, ENEMY_HEAL.
description	String	Texto descriptivo mostrado al jugador.

Metodos

Firma / Nombre	Descripcion
Event(EventType, String)	Constructor que asigna tipo y descripcion.
apply(RunManager)	Aplica el efecto: cura/envenena jugador, activa harder_combat, modificador global o vida extra al enemigo.

Juego

Juego.java

Vista principal del juego (hereda SurfaceView). Contiene toda la logica de pantallas (menu, seleccion de personaje, lore, combate, pausa, resultado, seleccion de buff y evento), manejo de entrada tactil multitoque, escalado del mundo (1920x1080) a cualquier resolucion, y gestion de musica. Orquestador central que conecta todos los sistemas.

Constantes

Nombre	Valor/Tipo	Descripción
WORLD_W / WORLD_H	float 1920/1080	Tamano fijo del mundo logico en pixeles.
INITIAL_TIME	float = 90	Segundos de partida en modo Arcade.
MOVE_SPEED	float = 650	Velocidad base de movimiento del jugador (unidades/seg).
MAP_MIN_X	float = 150	Limite horizontal minimo/maximo del escenario.
BAR_WIDTH / BAR_HEIGHT	float 700/45	Dimensiones de las barras de vida.
BAR_COMBO_H	float = 12	Altura de la barra de combo.
DEATH_ANIM_WAIT	float = 2.5	Segundos de espera tras la muerte antes del resultado.
HIT_FLASH_DURATION	float = 0.10	Duracion del efecto visual de impacto en segundos.

Variables y Campos

Nombre	Tipo	Descripción
activity	Activity	Referencia a la Activity para assets y SharedPreferences.
res	ResourceManager	Todos los assets visuales y escenarios.
prefs	SharedPreferences	Persistencia de puntuaciones y mute.
bucle	BucleJuego	Hilo del bucle de juego.
music	MediaPlayer	Reproductor de musica de fondo.
joystick	VirtualJoystick	Joystick tactil del jugador.
worldMatrix	Matrix	Transforma coordenadas mundo->pantalla.
inverseMatrix	Matrix	Transforma coordenadas pantalla->mundo (para touch).
activeSlot	int	Modo activo: >=0 Historia, -1 Duelo, -2 Arcade.
currentLevel	int	Nivel actual en modo Historia (1..MAX_LEVELS).
currentScreen	Screen	Pantalla activa: MAIN_MENU, CHAR_SELECT, LORE, FIGHTING, PAUSED, RESULT, BUFF_SELECT, EVENT.
player1 / player2	Player	Instancias de los dos combatientes.
enemyAI	EnemyAI	IA que controla a player2.

timeLeft	float	Tiempo restante en Arcade.
enemiesDefeated	int	Contador de enemigos eliminados en la partida Arcade actual.
totalTimeSurvived	float	Tiempo total sobrevivido en la partida Arcade actual.
runManager	RunManager	Gestiona buffs y escalado de enemigos en Arcade.
pendingResult	boolean	True mientras se espera que termine la animacion de muerte.
pendingResultWon	boolean	Resultado del combate que se mostrara tras el delay.
deathDelay	float	Acumulador de tiempo desde que se activo pendingResult.
lastBattleWon	boolean	Resultado del ultimo combate completado.
currentStage	StageDef	Escenario actualmente en uso.
selectedCharIdx	int	Indice del personaje seleccionado por el jugador.
selectedStageIdx	int	Indice del escenario seleccionado.
buffChoices	List<Buff>	Las tres opciones de buff generadas tras ganar en Arcade.
pendingEvent	Event	Evento especial pendiente de ser mostrado.
hitFlashTimer	float	Temporizador del efecto visual de impacto.
hitFlashX/Y	float	Posicion del efecto de impacto en coordenadas mundo.
btn* (RectF)	RectF	Rectangulos de colision de cada boton de la UI (uno por boton, sin reutilizar).
paintOverlay	Paint	Pincel negro semitransparente para overlays.
paintBlack	Paint	Pincel negro solido (fondos de barras).
paintBitmap	Paint	Pincel con filtro bilinear para bitmaps.
paintHitFlash	Paint	Pincel para el efecto circular de impacto.

Metodos

Firma / Nombre	Descripcion
Juego(Activity, ResourceManager, SharedPreferences)	Constructor: inicializa SurfaceView, crea joystick y posiciona todos los botones.
initAllButtonRects()	Calcula las posiciones fijas de todos los botones en coordenadas mundo.
surfaceCreated(holder)	Obtiene tamano de pantalla, crea BucleJuego e inicia musica.
surfaceChanged(h, f, w, h2)	Actualiza la matriz de escala cuando cambia el tamano de la Surface.
surfaceDestroyed(holder)	Para el BucleJuego.
updateScaleMatrix(w, h)	Recalcula worldMatrix e inverseMatrix y las pasa al joystick.

<code>stopBucle()</code>	Senala enEjecucion=false y espera a que el hilo termine (join).
<code>actualizar(float delta)</code>	Punto de entrada del update; delega en updateFighting si pantalla es FIGHTING.
<code>renderizar(Canvas)</code>	Aplica worldMatrix y despacha el dibujo segun la pantalla activa.
<code>updateFighting(float)</code>	Logica del combate: mueve player1 con joystick, actualiza jugadores e IA, gestiona hit flash y condiciones de victoria.
<code>triggerHitFlash(P1,P2)</code>	Posiciona y activa el efecto visual de impacto entre los dos personajes.
<code>triggerResult(boolean)</code>	Inicia secuencia de fin de combate: activa pendingResult, llama a startDeath/forceIdle.
<code>drawMainMenu(Canvas)</code>	Dibuja fondo del titulo y los tres botones de modo de juego.
<code>drawCharSelect(Canvas)</code>	Dibuja la seleccion de personaje y escenario.
<code>drawLore(Canvas)</code>	Dibuja el texto narrativo del nivel con wrap automatico.
<code>drawFighting(Canvas)</code>	Dibuja fondo con parallax, personajes, hit flash, overlay de especial, barras y botones.
<code>drawPause(Canvas)</code>	Overlay de pausa con botones Continuar y Salir.
<code>drawResult(Canvas)</code>	Overlay de resultado con texto y botones de accion.
<code>drawBuffSelect(Canvas)</code>	Pantalla de seleccion de buff con tres opciones.
<code>drawEvent(Canvas)</code>	Pantalla de evento especial con descripcion.
<code>drawBars(Canvas)</code>	Barras de vida y combo de ambos jugadores con nombres.
<code>drawBar(c,x,y,w,h,pct,c)</code>	Dibuja una barra individual (fondo negro + relleno proporcional).
<code>drawHUD(Canvas)</code>	Boton de pausa y, en Arcade, temporizador y contador de enemigos.
<code>drawAttackButtons(Canvas)</code>	Cuatro botones de accion en la esquina inferior derecha.
<code>onTouchEvent(MotionEvent)</code>	Distribuye evento al joystick y convierte coordenadas pantalla->mundo.
<code>handleTap(float wx, float wy)</code>	Gestiona toques segun la pantalla activa.
<code>startFight()</code>	Crea player1 y player2, configura IA y modo, inicia RunManager en Arcade.
<code>resolveEnemyName()</code>	Devuelve nombre del enemigo segun modo (nivel Historia o aleatorio).
<code>spawnNewEnemy()</code>	Recarga animaciones de player2 con nuevo personaje sin recrear el objeto.
<code>applyModeConfig()</code>	Ajusta stats de vida, velocidad, danyo y cadencia segun modo y nivel.
<code>showResult(boolean)</code>	Cambia a pantalla RESULT, incrementa contador Arcade y guarda puntuacion.
<code>onResultAction()</code>	Accion del boton principal del resultado: buff en Arcade, siguiente nivel en Historia.

<code>checkForEvent()</code>	Comprueba si toca evento especial (cada 3 victorias en Arcade).
<code>resetCombat()</code>	Reinicia posiciones, salud, combo y timers para el proximo combate.
<code>saveArcadeScore()</code>	Guarda puntuacion Arcade en el top-5 de SharedPreferences.
<code>startMusic()/pause/resume/release</code>	Gestion completa del ciclo de vida de la musica de fondo.
<code>drawButton(c, r, label, bg, f)</code>	Dibuja boton redondeado con fondo, borde y texto centrado.
<code>drawCenteredText(c, txt, y, f)</code>	Dibuja texto horizontalmente centrado en el mundo.
<code>drawMultilineText(...)</code>	Dibuja texto con saltos de linea manuales (<code>\n</code>).
<code>drawWrappedText(...)</code>	Dibuja texto con word-wrap automatico dentro de un ancho maximo.
<code>hit(RectF, float, float)</code>	Devuelve true si el punto (x,y) esta dentro del rectangulo r.

Player

Player.java

Representa a un personaje del juego. Gestiona animaciones por estado (idle, caminar, atacar, danyo, saltar, morir), fisica basica (gravedad, salto), hitboxes de cuerpo y ataque, stats de combate y dibujado del sprite con espejo horizontal. Incluye modo debug para visualizar hitboxes.

Constantes

Nombre	Valor/Tipo	Descripción
GRAVITY	float = -2500	Aceleracion gravitacional en unidades/seg ² .
JUMP_FORCE	float = 1150	Impulso vertical al saltar.
MAX_COMBO_CHARGE	float = 100	Carga maxima del medidor de combo para el especial.
DEBUG_HITBOXES	static bool	Activa/desactiva el dibujado de hitboxes en pantalla.

Variables y Campos

Nombre	Tipo	Descripción
walkAnim/idleAnim/punchAnim/kickAnim	SpriteAnimation	Animaciones de caminar, reposo, punyo y patada.
kickVisualAnim	SpriteAnimation	Visual del ataque kick (usa animacion de carrera si no hay sprites).
hurtAnim/jumpAnim/deathAnim	SpriteAnimation	Animaciones de danyo, salto y muerte.
currentState	State	Estado actual: IDLE, WALK, ATTACKING, HURT o DEAD.
currentAttackType	AttackType	Tipo de ataque en curso: PUNCH, KICK o SPECIAL.
stateTime	float	Tiempo acumulado en el estado actual.
attackTime	float	Tiempo transcurrido desde el inicio del ataque actual.
hurtTime	float	Tiempo transcurrido desde que empezo el estado HURT.
jumpTime	float	Tiempo en el aire (para sincronizar animacion de salto).
deathTime	float	Tiempo transcurrido desde el inicio de la animacion de muerte.
hasJumpAnim/hasKickAnim/hasDeathAnim/hasHurtAnim	boolean	Flags de animaciones opcionales disponibles.
x / y	float	Posicion del personaje en coordenadas fisicas (Y=0 es el suelo).
velocityY	float	Velocidad vertical actual (positivo=subir, negativo=caer).

groundY	float	Nivel del suelo especifico del personaje en este escenario.
scale	float	Factor de escala de dibujado (4.0 para todos los personajes actuales).
drawOffsetX/Y	float	Desplazamiento del sprite en canvas para alinear pies con groundY.
worldHeight	float	Altura del mundo logico (1080), para convertir Y fisica a Y canvas.
hbWMult/hbHMult	float	Multiplicadores de ancho y alto de la hitbox respecto al frame.
hbOffsetX/Y	float	Offsets en px canvas para centrar la hitbox sobre el personaje real.
name/charType/charPrefix	String	Nombre del personaje, tipo y prefijo de archivos de sprite.
maxHealth/currentHealth	float	Vida maxima y actual.
comboCharge	float	Carga actual del medidor de combo (0..MAX_COMBO_CHARGE).
damageMultiplier	float	Multiplicador global de danyo infligido.
speedMultiplier	float	Multiplicador de velocidad de movimiento.
lifestealPercent	float	Fraccion del danyo infligido que se recupera como vida.
armorPercent	float	Fraccion del danyo recibido que se absorbe.
criticalChance	float	Probabilidad (0..1) de golpe critico (x2 danyo).
regenPerSec	float	Vida recuperada por segundo de forma pasiva.
facingRight	boolean	Direccion en que mira el personaje (true=derecha).
opponent	Player	Referencia al oponente para calcular knockback y colisiones.
hasHitInCurrentAttack	boolean	Evita registrar mas de un golpe por ataque.
justHitThisFrame	boolean	True el frame en que se registro un golpe (para el hit flash).
paint/hurtPaint	Paint	Pinceles de dibujado: normal y con filtro rojo parpadeante.
matrix	Matrix	Matriz reutilizable para transformaciones del sprite.

Metodos

Firma / Nombre	Descripción
Player(name,assets,folder,prefix,x,y,facing,WH)	Constructor: inicializa campos, configura hurtPaint y carga animaciones.
loadAnimations(assets,folder,prefix,WH)	Carga todas las animaciones, determina cuales son opcionales y llama a calibrateScale.

<code>calibrateScale(String prefix)</code>	Ajusta scale, drawOffsetY, hbWMult, hbHMult, hbOffsetX/Y segun el personaje.
<code>getBodyCanvasY()</code>	Devuelve la Y canvas del centro del torso, para el efecto de impacto.
<code>update(float delta)</code>	Avanza fisica (gravedad, posicion), timer de danyo, regeneracion y stateTime.
<code>updateAttack(float delta)</code>	Avanza timer de ataque, aplica danyo/knockback/lifesteal al oponente en el momento correcto.
<code>jump()</code>	Aplica JUMP_FORCE si el personaje esta en el suelo y no esta en HURT.
<code>move(float amount)</code>	Desplaza el personaje en X si no esta atacando ni recibiendo danyo.
<code>updateDirection(boolean r, l)</code>	Actualiza facingRight y transita a IDLE cuando el jugador suelta el joystick.
<code>attack(AttackType)</code>	Inicia un ataque si el personaje esta libre; requiere combo lleno para SPECIAL.
<code>takeDamage(float, float)</code>	Aplica reduccion por armadura, descuenta vida, entra en HURT y aplica knockback.
<code>addCharge(float)</code>	Incrementa el medidor de combo hasta MAX_COMBO_CHARGE.
<code>forceIdle()</code>	Fuerza el estado IDLE reiniciando attackTime y hurtTime.
<code>startDeath()</code>	Inicia el estado DEAD deteniendo la fisica.
<code>getHitbox()</code>	Devuelve RectF en coordenadas canvas de la hitbox del cuerpo, aplicando offsets por personaje.
<code>canHit(Player other)</code>	Comprueba si el ataque alcanza la hitbox del oponente usando RectF.intersects().
<code>draw(Canvas)</code>	Dibuja el frame actual con la matriz correcta (espejo si mira izq.) y llama a drawDebugHitbox si DEBUG_HITBOXES.
<code>drawDebugHitbox(Canvas)</code>	Dibuja rectangulo verde de la hitbox del cuerpo y rojo del area de ataque.
<code>currentFrame()</code>	Selecciona y devuelve el Bitmap segun estado y tiempo actuales.
<code>isAttacking()/isHurt()/isDead()/isGrounded()</code>	Getters de estado del personaje.
<code>getCurrentAttackType()</code>	Devuelve el AttackType en curso o null.
<code>recycle()</code>	Libera todos los bitmaps de las animaciones.

ResourceManager

ResourceManager.java

Carga y centraliza todos los assets visuales al inicio. Proporciona fuentes (Paint), bitmaps de UI, joystick, iconos y los tres escenarios (StageDef). Equivalente al ResourceManager de libGDX usando APIs de Android.

Variables y Campos

Nombre	Tipo	Descripción
fontSmall/fontMedium/fontBig	Paint	Fuentes bold a 36sp, 52sp y 72sp para HUD, botones y títulos.
bgTitle	Bitmap	Imagen de fondo de la pantalla principal.
joystickBg	Bitmap	Imagen de la base del joystick.
joystickKnob	Bitmap	Imagen del knob (stick) del joystick.
soundOn/Off	Bitmap	Iconos de sonido activado/desactivado.
stages	StageDef[]	Array con los tres escenarios: Bosque, Desierto y Ciudad.
assets	AssetManager	AssetManager expuesto para que Player cargue sprites.

Metodos

Firma / Nombre	Descripción
ResourceManager(Context)	Crea Paint de fuentes, carga todos los bitmaps y construye los tres StageDef.
getFolderForChar(String)	Devuelve la ruta de la carpeta de sprites segun el nombre del personaje.
getPrefixForChar(String)	Devuelve el prefijo de los archivos de frame segun el personaje.
loadBitmap(String)	Carga un Bitmap desde assets sin escalar; devuelve pixel magenta si no existe.
makePaint(float, int)	(Privado) Crea un Paint con anti-alias, color, tamaño y fuente bold.
recycle()	Libera todos los bitmaps y llama a recycle() de cada StageDef.

RunManager

RunManager.java

Gestiona la progresion de una partida en modo Arcade (roguelite): lleva la cuenta de combates ganados, almacena buffs activos del jugador, genera opciones de mejora aleatorias con rareza, aplica el escalado progresivo de enemigos y dispara eventos especiales cada 3 victorias.

Constantes

Nombre	Valor/Tipo	Descripción
MAX_BUFF_SLOTS	int = 6	Numero maximo de buffs que puede acumular el jugador.

Variables y Campos

Nombre	Tipo	Descripción
player	Player	Referencia al personaje del jugador.
fightswon	int	Combates ganados en la partida actual.
activeBuffs	List<Buff>	Buffs aplicados al jugador.
harderCombatNext	boolean	El proximo enemigo sera 50% mas fuerte.
enemyHealNext	boolean	El proximo enemigo tendra vida extra.
globalDamageMultiplier	float	Multiplicador de danyo para la IA en el proximo combate.
pendingEnemyBuffs	List<Buff>	Buffs no elegidos que recibe el enemigo.

Metodos

Firma / Nombre	Descripción
RunManager(Player)	Constructor: inicializa contadores y listas.
setPlayer(Player)	Sustituye al jugador y re-aplica todos los buffs activos.
onFightWon()	Incrementa fightswon y reinicia los flags de combate especial.
shouldTriggerEvent()	Devuelve true cada 3 victorias para disparar un evento especial.
generateRandomEvent()	Crea y devuelve un Event aleatorio de entre los cinco tipos.
getRandomBuffChoices()	Genera tres buffs aleatorios con tipo, rareza y valor calculados al azar.
applyBuffToPlayer(Buff)	Aniade el buff a activeBuffs y aplica su efecto al Player.
addPendingEnemyBuff(Buff)	Aniade un buff rechazado para el enemigo.
reapplyBuffs()	(Priv.) Re-aplica todos los buffs activos tras cambiar de Player.
applyBuffDirectly(P, Buff)	(Priv.) Modifica el stat del Player correspondiente al tipo de buff.

<code>applyEnemyScaling(AI, P)</code>	Escala al enemigo segun flags de evento, modificador global y victorias.
<code>applyEnemyBuff(AI, P, type)</code>	(Priv.) Aplica un BuffType concreto a los stats del enemigo.
<code>getPlayer()/getFightsWon()/getActiveBuffs()</code>	Getters de estado.
<code>setHarderCombatNext(bool)/setEnemyHealNext(bool) /setGlobalDamageMultiplier(float)</code>	Setters para flags de eventos.

SpriteAnimation

SpriteAnimation.java

Sustituto de TextureAtlas+Animation de libGDX. Carga frames individuales (.png/.webp) desde assets ordenados por nombre y los reproduce con duracion configurable por frame y dos modos (LOOP y NORMAL/one-shot). Fallback de un pixel blanco para sprites inexistentes.

Variables y Campos

Nombre	Tipo	Descripción
frames	Bitmap[]	Array de fotogramas cargados en memoria.
frameDuration	float	Duracion en segundos de cada fotograma.
playMode	PlayMode	Modo de reproduccion: LOOP (bucle) o NORMAL (congela en ultimo frame).
totalDuration	float	Duracion total = frameDuration x numero de frames.

Metodos

Firma / Nombre	Descripción
SpriteAnimation(Bitmap[], float, PlayMode)	Constructor que calcula la duracion total.
getKeyFrame(float stateTime)	Devuelve el Bitmap del frame segun el tiempo (con o sin bucle).
isFinished(float stateTime)	True si la animacion NORMAL ha completado todos sus frames.
getDuration()	Devuelve la duracion total de la animacion.
getFrameWidth()/getFrameHeight()	Ancho/alto en pixeles del primer frame (o 1 si no hay frames).
recycle()	Libera todos los Bitmaps de la memoria.
load(assets, folder, prefix, dur, mode)	(Static) Busca archivos en la carpeta, los ordena y construye la animacion.
fallback(float, PlayMode)	(Static) Devuelve animacion de un solo pixel blanco.

StageDef

StageDef.java

Define un escenario de combate: nombre, bitmap de fondo, offset visual del suelo y la posicion Y de groundY para los personajes. Permite registrar offsets Y individuales por personaje para ajustar la posicion inicial de cada uno.

Variables y Campos

Nombre	Tipo	Descripcion
name	String	Nombre del escenario mostrado en la seleccion.
bgBitmap	Bitmap	Imagen de fondo del escenario.
floorVisualOffset	float	Desplazamiento visual del suelo (ajuste estetico).
groundY	float	Posicion Y fisica del suelo para los personajes.
charYOffsets	Map<String, Float>	Offsets Y individuales por nombre de personaje.

Metodos

Firma / Nombre	Descripcion
StageDef(String, Bitmap, float, float)	Constructor que asigna todos los campos base.
addOffset(String charName, float)	Registra un offset Y para un personaje concreto (encadenable).
getOffsetFor(String charName)	Devuelve el offset Y del personaje o o si no tiene.
recycle()	Libera el bitmap de fondo.

VirtualJoystick

VirtualJoystick.java

Joystick tactil dibujado sobre el Canvas. Soporta multitoque (rastrea el puntero específico que lo activo), convierte coordenadas pantalla->mundo con matriz inversa, limita el knob al radio de la base y devuelve desplazamiento normalizado (-1..+1) en ambos ejes.

Variables y Campos

Nombre	Tipo	Descripción
centerX/centerY	float	Centro fijo de la base en coordenadas mundo.
radius	float	Radio de la base del joystick.
knobRadius	float	Radio del knob = 70% del radio de la base.
knobX/knobY	float	Posición actual del knob (vuelve al centro al soltar).
trackingPointerId	int	ID del puntero táctil activo (-1 si ninguno).
bgBitmap/knobBitmap	Bitmap	Imágenes de la base y el knob.
paint	Paint	Pincel con filtro bilinear para el dibujo.
inverseMatrix	Matrix	Transforma coordenadas pantalla->mundo para el touch.

Metodos

Firma / Nombre	Descripción
VirtualJoystick(cx, cy, r, bg, knob)	Constructor: calcula knobRadius y centra el knob.
setInverseMatrix(Matrix)	Actualiza la matriz de conversión pantalla->mundo.
onTouchEvent(MotionEvent)	Gestiona DOWN (activa tracking), MOVE (mueve knob) y UP/CANCEL (suelta y recentra).
updateKnob(float, float)	(Privado) Nueva posición del knob limitada al radio de la base.
isInsideBase(float, float)	(Privado) True si el punto está dentro del área de la base (margen +30%).
getKnobPercentX()	Desplazamiento X normalizado: -1.0 (izquierda) a +1.0 (derecha).
getKnobPercentY()	Desplazamiento Y normalizado: -1.0 (arriba) a +1.0 (abajo).
isActive()	True si hay un puntero táctil controlando el joystick.
draw(Canvas)	Dibuja base y knob como bitmaps escalados a sus radios.